
Climate Cognitive System

Gestión climática predictiva para aulas universitarias

Curso: Computación Cognitiva

Docente: Farfán Madariaga, Jaime Moshe

Institución: UTEC — Barranco, Lima

Integrantes del grupo

Balcazar Santa Cruz, Kiara Alexandra

Gonzales Torres, Aaron Naif

Surco Vergara, Maria Fernanda

Índice

1. Problema identificado	2
2. Propuesta de solución	2
3. Perfiles del sistema	2
4. Aplicación web	3
5. Hardware a utilizar	3
6. Uso de Inteligencia Artificial	4
7. Arquitectura inicial de la solución	5
8. Tecnologías propuestas	5
9. Alcance inicial del proyecto	5
10. Viabilidad del proyecto	6
11. Impacto esperado	6
12. Recursos y enlaces del proyecto	6

1. Problema identificado

En los entornos universitarios, los sistemas de aire acondicionado (A/C) de las aulas operan tradicionalmente en modo *always-on* durante todo el horario laboral, sin considerar la ocupación real del espacio ni las condiciones ambientales del momento.

¿Qué problema existe? El A/C enfría aulas vacías o con baja asistencia con la misma intensidad que aulas llenas, porque el control se basa en el horario y no en la ocupación efectiva. Esto genera un consumo energético sistemáticamente mayor al necesario.

¿A quién afecta? A la institución (UTECS), que asume el sobre costo energético; al personal de mantenimiento, que carece de datos para optimizar; y al medio ambiente, por la huella de carbono asociada al consumo eléctrico.

¿Por qué es importante resolverlo? Los sistemas HVAC representan cerca del 50 % del consumo energético de un edificio no residencial [3], la mayor fracción individual del gasto eléctrico de un campus. Ajustar la demanda a la ocupación real reduce costos y emisiones sin sacrificar el confort térmico recomendado ($\sim 20\text{--}24^\circ\text{C}$ según ASHRAE 55 [1]).

¿Qué ocurre actualmente si no se soluciona? Se mantiene el desperdicio energético, no existe trazabilidad del uso real de los espacios, y las decisiones de climatización siguen siendo manuales y reactivas en lugar de anticipatorias.

2. Propuesta de solución

Climate Cognitive System (CCS) es una plataforma IoT de gestión climática *predictiva* que ajusta dinámicamente el *setpoint* del A/C combinando ocupación real, condiciones ambientales y un motor cognitivo de Machine Learning con *fallback* heurístico.

¿Cómo funcionará? Sensores físicos reportan temperatura, humedad, CO_2 y CO al backend. El sistema estima la **ocupación real** a partir del CO_2 (balance de masa) y la contrasta con la **ocupación esperada** de la reserva. Con ambas señales, más el clima exterior, un modelo predictivo calcula cuántos grados pre-compensar y emite una acción cognitiva sobre el A/C (ON/STANDBY).

¿Qué valor aporta? Enfría según quién *está* en el aula, no según quién fue reservado: un aula medio vacía produce poco CO_2 , el sistema lo detecta y reduce la demanda, ahorrando energía. Además calcula el **ROI energético** frente al sistema tradicional.

¿Qué lo diferencia de una solución tradicional? El control clásico es *always-on* por horario. CCS es *feed-forward + feedback*: anticipa con la reserva y corrige en tiempo real con la ocupación medida, integrando además un asistente conversacional (IBM Watson) y detección de emergencias por monóxido de carbono.

3. Perfiles del sistema

El sistema implementa control de acceso por roles (RBAC) de tres niveles.

Perfil	Acciones principales
Usuario (<code>guest</code> / <code>collaborator</code>)	Registrarse e iniciar sesión; consultar el estado de aulas y sensores en tiempo real; crear reservas; recibir alertas de emergencia (CO); consultar el asistente conversacional; el <code>collaborator</code> puede además controlar el sensor de un aula <i>si tiene una reserva activa</i> .
Administrador (<code>admin</code>)	Gestionar usuarios (aprobar, rechazar, reactivar); aprovisionar y configurar aulas y sensores; supervisar la telemetría recolectada; administrar reservas; revisar reportes de ROI energético; recargar el modelo de IA.

Tabla 1: Perfiles obligatorios y sus funciones.

Las cuentas `guest` se auto-aprueban; `collaborator` y `admin` quedan en estado `pending` hasta aprobación manual del administrador.

4. Aplicación web

Se desarrolla una **aplicación web responsive** (React + Vite + TypeScript), pensada como panel administrativo y de monitoreo. Módulos principales:

- **Inicio de sesión / Registro** — autenticación JWT y flujo de aprobación por rol.
- **Dashboard global** — monitor de telemetría en vivo.
- **Aulas y detalle de aula** — estado del A/C en tiempo real, historial de lecturas, control físico del sensor.
- **Reservas** — CRUD con validación de horario.
- **Infraestructura** — panel dual Aulas ↔ Sensores para aprovisionamiento.
- **Reporte de ROI** — KPIs energéticos y comparación Tradicional vs. Cognitivo.
- **Gestión de usuarios** — aprobación de cuentas (solo admin).
- **Chat flotante** — asistente IBM Watson embebido.

Cada módulo respeta el RBAC: el menú se filtra según el rol autenticado, en espejo con la autorización del backend.

5. Hardware a utilizar

La telemetría se captura con sensores de bajo costo sobre un **Arduino UNO** que emite las lecturas por puerto serial a un equipo puente. En la versión actual se cuenta con un nodo sensor funcional instalado en el aula A403 — el script del gateway está versionado en el repositorio (`hardware/sistema.py`) y su montaje se evidencia en los videos de demostración enlazados en §12 —; el resto del despliegue se modela mediante un simulador y un dataset sembrado reproducible (ver §10).

Dispositivo	Función	Datos que captura	Frecuencia
DHT11	Ambiente térmico	Temperatura (°C), humedad (%)	~2 s
MQ-135	Calidad de aire	CO ₂ (ppm, proxy) → ocupación real	~2 s
MQ-7	Seguridad	CO (ADC → ppm, curva calibrada)	~2 s
Cámara + MediaPipe	Aforo	Conteo de personas por cruce de línea	tiempo real
Emisor IR*	Actuador	Comando de <i>setpoint</i> al A/C	bajo demanda
Arduino UNO	Nodo sensor	Agrega lecturas y las emite por Serial (JSON)	~2 s

Tabla 2: Hardware, función, datos y frecuencia de envío.

* Actuador de fase 2: en la versión inicial la acción cognitiva (ON/STANDBY) se registra y visualiza en la aplicación; el envío IR real al A/C es funcionalidad opcional (§9).

Conexión con la aplicación. El Arduino UNO emite cada ~2 s un JSON por Serial (9600 baud) con las lecturas de los tres sensores. Un script Python en el equipo puente (`hardware/sistema.py`) parsea la trama, convierte la lectura ADC del MQ-7 a ppm reales mediante la curva del *datasheet*, fusiona el aforo estimado por la cámara (MediaPipe) y publica `POST /api/v1/sensors/` en formato JSON (`sensor_id`, `temperature`, `humidity`, `co2_ppm`, `co_ppm`) por HTTPS a la instancia desplegada, autenticado por `X-API-Key`. El backend persiste la lectura y responde la acción cognitiva; el actuador IR recibirá el *setpoint* sugerido. El umbral de CO > 50 ppm dispara alertas de emergencia [6].

6. Uso de Inteligencia Artificial

La IA es el núcleo funcional, no un adorno. Opera en varias capas:

- **Predicción de carga térmica (ML).** Un `HistGradientBoostingRegressor` [5] predice cuántos grados pre-compensar, a partir de temperatura, hora, ocupación esperada, ocupación real, temperatura exterior y metadata física del aula. Se valida con *holdout* cronológico frente a dos *baselines* (media y heurística de ocupación); el modelo solo se despliega si las supera (`beats_baselines`).
- **Estimación de ocupación real.** Balance de masa sobre el CO₂ [2]: $personas \approx (CO_2 - base) / ppm\text{-por-persona}$.
- **Fallback heurístico.** Si no hay modelo entrenado, una regla determinística mantiene el servicio operativo.
- **Detección de anomalías.** Umbrales sobre temperatura, humedad y CO para emergencias.
- **Visión por computadora (aforo).** MediaPipe Face Detection en el nodo cliente cuenta entradas y salidas por cruce de línea virtual; el aforo se registra para trazabilidad y validación de la estimación por CO₂.
- **Asistente conversacional (NLP).** IBM Watson Assistant vía *custom extension* sobre la API.
- **Reportes automáticos.** Cálculo de ROI energético (kWh y CO₂ evitados).

Datos, resultado y beneficio. Procesa la telemetría histórica de los sensores más el contexto de reservas; produce una decisión de setpoint y un reporte de ahorro; el beneficio es doble: energía/costo para el administrador y confort estable para el usuario.

7. Arquitectura inicial de la solución

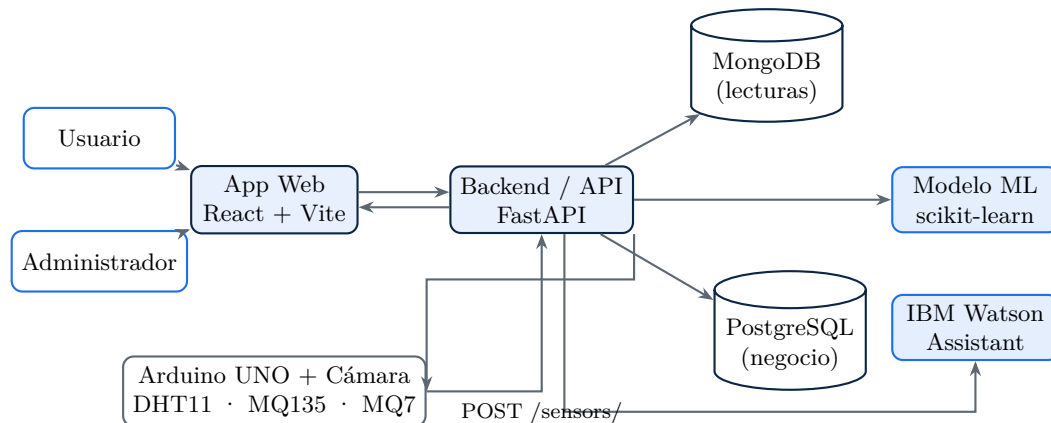


Figura 1: Arquitectura: usuarios y administrador operan la app web, que consume el backend FastAPI; éste persiste en MongoDB/PostgreSQL, invoca el modelo ML y actúa como *proxy* de IBM Watson, y recibe la telemetría del hardware IoT.

8. Tecnologías propuestas

Capa	Tecnologías
Frontend	React 18, Vite, TypeScript, Tailwind CSS, Recharts
Backend	Python, FastAPI [4], SQLAlchemy, Uvicorn; JWT + Argon2 (pwdlib)
Bases de datos	PostgreSQL 15 (negocio), MongoDB 6 (telemetría)
IA	scikit-learn (HistGradientBoostingRegressor), IBM Watson Assistant, MediaPipe (visión)
Hardware	Arduino UNO, DHT11, MQ-135, MQ-7, cámara + MediaPipe, emisor IR
Despliegue	Docker + Docker Compose, Nginx

Tabla 3: Stack tecnológico por capa.

9. Alcance inicial del proyecto

Funcionalidades obligatorias

- Ingesta de telemetría y persistencia.
- Motor cognitivo (ML + heurístico) con acción sobre el A/C.
- Estimación de ocupación real vía CO₂.
- RBAC de 3 roles + autenticación JWT.
- App web con dashboards, reservas e infraestructura.
- Detección y alerta de emergencias por CO.
- Reporte de ROI energético.

Funcionalidades opcionales

- Migración a la nube (AWS/GCP).
- Reentrenamiento automático del modelo.
- Control real del A/C por emisor IR calibrado.
- Notificaciones por correo.
- Exportación de reportes a PDF.
- Integración de API meteorológica real.

10. Viabilidad del proyecto

Factor	Evaluación
Hardware	Sensores de bajo costo y ampliamente disponibles; un nodo funcional ya operativo.
Herramientas de IA	scikit-learn e IBM Watson accesibles sin costo relevante.
Conocimientos	Equipo con dominio de Python, React y bases de datos.
Tiempo	Backend, frontend y motor cognitivo ya implementados en su primera versión.
Riesgo principal	Disponibilidad limitada de hardware (un solo sensor).
Mitigación	Simulador de telemetría + dataset sembrado reproducible (<code>seed_data/</code>), que permite validar todo el pipeline sin múltiples sensores.

Tabla 4: Análisis de viabilidad y gestión de riesgos.

11. Impacto esperado

- **Ambiental.** Reducción del consumo eléctrico del A/C y, por tanto, de las emisiones de CO₂ asociadas, al climatizar según ocupación real en lugar de un horario fijo.
- **Económico.** Ahorro directo en la factura energética del campus, cuantificado por el módulo de ROI.
- **Educativo / Institucional.** Trazabilidad del uso real de las aulas, útil para la planificación de espacios de UTEC.
- **Seguridad.** Detección temprana de niveles peligrosos de monóxido de carbono con alertas en tiempo real.
- **Automatización.** Sustituye el control manual y reactivo por decisiones anticipatorias basadas en datos.

12. Recursos y enlaces del proyecto

- **Repositorio de código (GitHub):**
<https://github.com/Kiarosaurus/Climate-Cognitive-System>
- **Instancia desplegada (producción):**
<https://kiarosaurus.me>
- **Videos de demostración (Software y Hardware) — Google Drive:**
<https://drive.google.com/drive/folders/1n6j4DU4aDTeXRAdlSHInQun2IR3O-uuh>

Referencias

- [1] ASHRAE. Standard 55 – Thermal Environmental Conditions for Human Occupancy, 2020.
- [2] Piotr Batog and Marek Badura. Dynamic of changes in carbon dioxide concentration in bedrooms, 2013.
- [3] Luis Pérez-Lombard, José Ortiz, and Christine Pout. A review on buildings energy consumption information. *Energy and Buildings*, 40(3):394–398, 2008.
- [4] Sebastián Ramírez. FastAPI, 2024.
- [5] scikit-learn developers. HistGradientBoostingRegressor – scikit-learn documentation, 2024.
- [6] U.S. Environmental Protection Agency. Carbon monoxide (CO) standards and health effects, 2023.

La documentación técnica extensa (arquitectura interna, decisiones de diseño y detalle del pipeline de ML) se encuentra en el anexo `TECHNICAL_DOCUMENTATION.md` del repositorio.